CVRadar v3

AI-Powered Resume Screening Matrix

User Guide & GitHub Documentation

Author: Sobhan Mohanty | Licence: MIT (Open Source)

⚠ DISCLAIMER

CVRadar is an experimental, open-source AI-assisted pipeline intended solely for preliminary CV screening. All results must be validated by a qualified human recruiter before any hiring decision is made. The author and contributors accept no responsibility or liability whatsoever for: (a) API costs incurred, including charges from failed runs, 503 / rate-limit errors, or model unavailability; (b) unsatisfactory, incorrect, or biased screening results; (c) any direct, indirect, or consequential loss arising from use of this software. By using CVRadar you agree that use is entirely at your own risk and discretion. This tool does not constitute legal, HR, or professional recruitment advice.

1. What is CVRadar?

CVRadar is a multi-provider AI resume screening application built with Streamlit. It accepts a Job Description (JD) and any number of candidate CVs, evaluates each CV against the JD, and produces a ranked Excel report with scores, fit categories, skills gaps, and analytics charts — all in a single pipeline run.

CVRadar is provider-agnostic: it supports Google Gemini, Anthropic Claude, and OpenAI GPT. It also includes a fully offline Track 1 (Local Keyword) engine that requires no API key and consumes zero tokens.

2. System Architecture

2.1 Three Processing Tracks

Track 1 Local Keyword	100% offline, zero API cost. Extracts skills, education, experience and domain from CV text using regex and a global domain taxonomy file. Best for fast, free pre-screening of large batches.
Track 2 LLM Semantic	One Gemini / Claude / OpenAI call per CV. Deep contextual reading — understands implied skills, career transitions, synonyms, and complex eligibility rules. Filter detection (Priority / Exclusion) is merged into the same call, so cost stays at one API request per CV regardless of whether filters are active.
Hybrid Pre-Filter → LLM	Track 1 runs first and scores every CV locally. Only CVs that clear a configurable threshold (default 40/100) are escalated to the LLM.

	Typically cuts API spend by 40–70% on noisy applicant pools. On LLM failure the local score is preserved (graceful degradation — no blank rows).

2.2 Scoring — Hard Python, Not AI

To prevent score hallucination, the LLM never generates numbers. Every CV — regardless of track — flows through a deterministic Python scorer with fixed weights:

Skills Match	35 points — matched must-have requirements as a percentage
Experience	20 points — extracted years vs JD requirement band
Education	15 points — degree level vs JD minimum
Projects	15 points — number of relevant projects found
Domain Expertise	10 points — domain keyword match against JD domain field
Certifications	5 points — matched certification keywords

2.3 Filter Layer (Optional)

After scoring, an optional filter layer applies flat adjustments:

- Priority trait detected → +15 points (bonus)
- Exclusion trait detected → −30 points (penalty)

Score is clamped to 0–100. Fit category is recomputed. The AI only detects whether a trait is present; the arithmetic is pure Python. Both tracks support the filter layer — Track 1 uses keyword matching; Track 2 uses LLM detection merged into the main evaluation call.

2.4 Fit Categories

Exceptional Fit	85–100 — strong shortlist candidate
Strong Fit	70–84 — shortlist candidate
Moderate Fit	55–69 — review recommended
Potential Fit	40–54 — borderline, human review needed
Low Fit	0–39 — does not meet core requirements

2.5 Reliability & Safety

- 45-second per-request timeout on all LLM calls
- 5 retry attempts with exponential backoff capped at 30 seconds
- Transient errors (503, 429, overload) retried; permanent errors fail fast
- Provider health check fires before the main loop — bad keys surface immediately
- Graceful degradation: LLM failure → Track 1 local score (no blank rows)
- Adaptive inter-request pacing (configurable 0.5–5 sec) to reduce rate-limit hits
- Checkpoint recovery: per-session, per-track ledger skips already-processed CVs on re-run

- Session-isolated storage: every browser session gets a UUID — concurrent users cannot see each other's data
- PII masking: emails and phone numbers are replaced before any text reaches the LLM

---

3. Installation

3.1 Prerequisites

- Python 3.10+
- pip (Anaconda or standard)
- Git

3.2 Clone and Install

```
git clone https://github.com/YOUR_USERNAME/CVRadar.git
cd CVRadar
pip install -r requirements.txt
```

3.3 requirements.txt (minimum)

```
streamlit
pandas
plotly
openpyxl
python-docx
PyPDF2
python-dotenv
pydantic
google-genai
anthropic
openai
```

3.4 Project Structure

```
CVRadar/
├── app.py                  ← main Streamlit application
├── config.py               ← scoring weights, fit categories, paths
├── .env                    ← API keys (never commit this file)
├── requirements.txt
└── src/
    ├── parser.py           ← PDF / DOCX text extraction + PII masking
    ├── jd_parser.py        ← local DOCX JD keyword extraction (Track 1)
    ├── llm_provider.py     ← unified Gemini / Claude / OpenAI adapter
    ├── evaluator.py        ← LLM evaluation schema and call logic
    ├── scorer.py           ← deterministic Python scorer
    ├── filters.py          ← priority / exclusion filter engine
    ├── reporting.py        ← Excel export and analytics dataframe
    ├── schemas.py          ← Pydantic data models
    ├── checkpoint_manager.py
    ├── status_tracker.py
    └── domain_taxonomy.txt ← global academic domain synonym list
```

3.5 Environment Setup

Create a .env file in the project root. Add only the keys for providers you intend to use:

```
# .env  — DO NOT commit this file to Git
GEMINI_API_KEY=your_gemini_key_here
ANTHROPIC_API_KEY=your_claude_key_here
OPENAI_API_KEY=your_openai_key_here
```

 Add .env to your .gitignore before the first commit. If deploying to Streamlit Cloud, add keys to Settings → Secrets instead of .env.

3.6 Run

```
cd CVRadar
streamlit run app.py
```

---

4. Job Description (JD) — Format Guide

The JD is the single most important input. CVRadar extracts skill keywords, experience requirements, education level, and domain from it. A well-structured JD produces accurate skill matching; a poorly formatted one produces broad or meaningless matches.

4.1 File Format

- Upload as .docx (Microsoft Word format) only
- PDF JDs are not accepted — PDF text extraction is fragile and produces garbled output
- Plain text (.txt) is not accepted

 If your JD exists as a PDF, open it in Word and Save As .docx before uploading.

4.2 Recommended JD Structure

CVRadar's local JD parser recognises section headers and extracts bullets under them. The more clearly your JD is sectioned, the better the keyword extraction. The ideal structure is:

```
Mandatory Technical Skills (Must-Haves)
• QGIS
• ArcGIS
• Remote Sensing
• Python (pandas, geopandas)
• Image classification

Preferred Skills (Nice-to-Have)
• Google Earth Engine (GEE)
• GDAL
• Deep learning for remote sensing

Education
Master's degree in Remote Sensing, Geoinformatics, Geography, or related field

Experience
2 to 3 years of professional working experience in Geospatial projects
```

4.3 Section Headers CVRadar Recognises

Must-Have section	Must-Have, Mandatory, Required Skills, Core Skills, Essential, Technical Skills, Minimum Qualifications
Nice-to-Have section	Nice-to-Have, Preferred, Good to Have, Desirable, Additional Skills, Added Advantage, Bonus
Education section	Education, Qualification, Degree, Academic
Experience section	Experience, Work Experience, Professional Experience, Years of
Domain section	Domain, Sector, Industry, Field of
Certifications	Certifications, License, Accreditation

4.4 How Skills Are Extracted

CVRadar extracts skills in two ways depending on your JD format:

- Bullet-point items: each bullet is taken as a single skill keyword. This is the most reliable format.
- Plain sentences: the parser extracts known technical terms from sentences like "Proficiency in QGIS, Erdas Imagine, and ArcGIS" → extracts [QGIS, Erdas Imagine, ArcGIS].

Avoid vague sentences like 'Must have a great understanding of geospatial concepts' — these produce no extractable keywords. Use specific tool and skill names instead.

4.5 Good vs Poor JD Examples

✓ Good — bullet points with specific tools	✗ Poor — vague sentences
• QGIS • ArcGIS • Remote Sensing • Python (pandas, geopandas)	Must have great understanding of geospatial concepts and tools for data analysis and mapping solutions in various environments.

4.6 Track 1 vs Track 2 JD Handling

Track 1 (Local)	JD file is parsed offline by jd_parser.py. The parser extracts bullet keywords and known technical terms from sentences. The resulting keyword list is matched word-by-word against each CV.
Track 2 / Hybrid	The raw JD text is sent verbatim to the LLM alongside each CV. The LLM reads both and evaluates contextual fit — so even non-bulleted JDs work well on Track 2.

For Track 1 accuracy, always use bullet-point format. For Track 2, natural prose JDs also work well.

5. Priority & Negative Filters

Filters are an optional scoring layer that runs after the base score is calculated. They let you reward candidates with specific desirable traits (+15 pts) and penalise candidates with disqualifying traits (−30 pts). Both adjustments are hard Python arithmetic — the AI only detects whether a trait is present.

5.1 Priority Traits (+15 points)

Use the Priority box to reward candidates who have specific strengths beyond the core JD.

Good priority trait examples:

ISRO background, NRSC experience	Rewards candidates with government remote sensing institute experience
peer-reviewed publications	Rewards candidates with academic research output
Google Earth Engine, GEE	Rewards specific tool experience beyond the must-haves
open-source contributions	Rewards candidates who contribute to open source
crop classification, SAR analysis	Rewards specific sub-domain expertise
hyperspectral, LiDAR	Rewards advanced sensor experience

5.2 Exclusion Traits (−30 points)

Use the Exclusion box to penalise candidates who clearly do not match your requirements. CVRadar supports three types of exclusion rules:

Type A — Experience thresholds (numeric)

freshers not required	Excludes candidates with < 1 year experience
less than 2 years	Excludes candidates with < 2 years experience
experience more than 7 years	Excludes over-experienced candidates (> 7 years)
under 3 years	Excludes candidates with < 3 years experience
1 to 5 years	Excludes candidates within that experience range
entry level	Excludes candidates with < 2 years experience

Type B — Role / title keywords

Business Analyst	Penalises CVs where 'Business Analyst' appears as a role or title
Data Analyst not required	Same as above — 'not required' is stripped and 'Data Analyst' is matched
Software Developer	Penalises pure software profiles
BPO, voice support	Penalises non-technical backgrounds
academia only	Penalises purely academic profiles with no industry experience

Type C — Career stage keywords

intern	Penalises candidates who are or were only interns
entry-level	Maps to experience < 2 years
trainee	Maps to experience < 2 years
director level	Maps to experience > 12 years (over-qualified)

 Only use job-relevant criteria in filters. Filters based on age, gender, religion, caste, nationality, marital status, disability, or any protected characteristic are discriminatory and must not be used under any circumstances.

5.3 Filter Syntax Rules

- Separate multiple traits with commas: freshers not required, Business Analyst, less than 2 years
- Order does not matter
- Case-insensitive: 'GEE experience' and 'gee experience' are equivalent
- Leave either box blank to skip that filter entirely
- Both boxes can be filled simultaneously

5.4 Full Example

```
Priority Traits:
GEE experience, ISRO background, peer-reviewed publications, crop classification

Exclusion Traits:
freshers not required, Business Analyst, Data Analyst not required,
experience more than 7 years
```

Effect: A candidate with GEE experience starts with base score + 15. A candidate who is a Business Analyst loses 30 points. A candidate with 8 years experience also loses 30 points (over-qualified for this JD). A fresher (< 1 year) loses 30 points.

---

6. Running a Screening

Step 1 — Sidebar Configuration

- Select LLM Provider (Gemini / Claude / OpenAI)
- Paste your API key if the server has no built-in key
- Select Model (Haiku / Flash for cost-efficient runs; Pro / Sonnet / GPT-4o for deeper accuracy)
- Select Processing Engine (Track 1 for free runs; Track 2 for full AI; Hybrid for balanced)
- Keep Duplicate Detection and Checkpoint Recovery checked

Step 2 — Upload JD

- Click 'Upload Job Description (.docx)' and select your JD file
- Expand 'Parsed JD Requirements' after uploading to verify the extracted keywords
- If must-have skills look wrong, improve the JD formatting and re-upload

Step 3 — Optional Filters

- Fill Priority Traits and/or Exclusion Traits as described in Section 5
- Leave both blank to skip the filter layer

Step 4 — Upload CVs

- Upload all candidate CVs (PDF or DOCX, multiple files)
- The upload panel accepts hundreds of files simultaneously

Step 5 — Execute

- Check both acknowledgement boxes
- Click '□ Execute CVRadar Pipeline'
- Monitor the progress bar and per-CV status messages

Step 6 — Review Results

- Inspect the summary metrics (Total, Avg Score, Highest, Shortlisted)
- Review the Stream & Education Analytics charts
- Examine Skills Matched and Skills Missing charts to understand the talent pool
- Download the Excel report for detailed per-candidate review

Batch Strategy for 300+ CVs

- Split uploads into 100–150 CVs per run (browser memory constraint)
- Download the Excel after each batch
- Upload all Excel files to Batch Analytics to get a merged cross-batch report
- Checkpoint Recovery automatically skips already-processed CVs on the next run

□ *On Track 2 with 341 CVs at 1.5 sec inter-request delay, expect approximately 20–25 minutes total runtime. Claude Haiku 4.5 costs approximately $1–1.50 for a full 341-CV run.*

---

7. Excel Output

The downloaded report contains one sheet (CVRadar_Results) with one row per candidate, ranked by Final Score descending. Column widths are auto-fitted. The header row is dark navy with white bold text. The Fit Category column is colour-coded.

Rank	Candidate rank by Final Score (1 = highest)
File Name	Full original filename (no truncation)
Candidate Name	Extracted from CV text
Primary Domain	Academic / professional stream detected from CV

Experience (Yrs)	Total professional experience in years
Education	Degree level (Track 1) or full degree sentence (Track 2)
Skills Matched	JD must-have skills found in the CV (semicolon separated)
Skills Missing	JD must-have skills NOT found in the CV
Projects	Top 3–5 project titles extracted from the CV
Final Score	0–100 deterministic score after filter adjustments
Fit Category	Exceptional / Strong / Moderate / Potential / Low Fit
Processing Track	Track 1 Local / Track 2 (Claude) / Degraded to Local / Hybrid-Culled
Filter Note	Explanation of any priority/exclusion adjustment applied

---

8. Contributing & Open Source

CVRadar is released under the MIT Licence. You are free to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the software, subject to the condition that the original copyright notice and this permission notice appear in all copies.

Contributing Guidelines

- Fork the repository and create a feature branch
- Follow the existing module structure — new LLM providers go in src/llm_provider.py
- Domain taxonomy additions go in src/domain_taxonomy.txt following the GROUP | Canonical | synonym1, synonym2 format
- Do not commit .env files, API keys, or candidate data
- Open a Pull Request with a clear description of the change

Extending the Domain Taxonomy

src/domain_taxonomy.txt uses a simple pipe-delimited format:

```
# Format: GROUP | Canonical Name | synonym1, synonym2, synonym3
GEOSPATIAL | GIS | gis, geographic information system, qgis, arcgis, geomatics
ENGINEERING | Civil Engineering | civil engineering, structural, geotechnical
LAW | Law | law, llb, llm, legal studies, jurisprudence
```

Add new streams or additional synonyms to improve Track 1 domain detection. Groups are used for chart-level aggregation. Synonyms are matched with word-boundary regex so partial matches are prevented.

---

9. Troubleshooting

0 candidates processed	Most likely cause: Checkpoint Recovery has all hashes from a previous run. Click Reset This Session and re-run. Note: each engine track has its own checkpoint ledger so switching from Track 1 to Track 2 is safe.
Streams show Other / Undetected	domain_taxonomy.txt is missing from the src/ folder. Copy it from the repository. Also verify by checking if the file is > 10 KB. Track 2 domain detection is handled by the LLM and is not affected by this file.
503 / rate-limit errors	Increase the Inter-request delay slider in the sidebar (try 3.0 sec). The retry engine will automatically retry up to 5 times with exponential backoff before marking a CV as failed.
Education pie shows full sentences	This occurs on Track 2 runs with older app.py. Upgrade to v3 which includes the normalise_education_level() function that maps full sentences to clean labels.
JD skills not matching CVs	Expand 'Parsed JD Requirements' in the app after uploading your JD. If must-have skills shows full sentences instead of short keywords, restructure the JD to use bullet points with specific tool names. See Section 4.
st.secrets error on local run	This is harmless on local machines. The app falls back to .env automatically. Ensure your .env file exists and contains the correct key names.
App shows Nexus v2 instead of CVRadar	The old app.py was not replaced. Verify by opening app.py and checking line 3 reads 'CVRadar v3'. If not, copy the new file from the repository.

10. Licence